Clautolisp User Manual

Codex

May 24, 2026

Contents

1 Scope

This manual documents the `clautolisp` standalone executable: how to invoke it, what options it accepts, and what it does in each mode.

It does **not** document the AutoLISP / Visual LISP language. For the language reference — surface syntax, builtins, semantics, and host-product divergences — see the companion `autolisp-spec` subproject (sibling directory `../../autolisp-spec/`).

This document must be kept in sync with the implementation. Whenever `clautolisp/tools/clautolisp/source/main.lisp` changes its option parsing, banner, exit codes, or supported modes, this file must be updated in the same commit.

The rendered PDF (`user-manual.pdf`, sibling of this file) is also checked in. Whenever this Org file changes, run `make user-manual` (or `make documentation`) to regenerate the PDF and stage both files in the same commit.

2 Synopsis

```
clautolisp [options] FILE.lsp
clautolisp [options] -x EXPRESSION
clautolisp [options]                 (interactive REPL)
```

The implementation is shipped as two image-built executables, one per supported Common Lisp host:

- `clautolisp-sbcl` — built on SBCL via `make build-clautolisp-sbcl`.
- `clautolisp-ccl` — built on CCL via `make build-clautolisp-ccl`.

Both binaries are produced under `clautolisp/tools/clautolisp/bin/` and accept the same command-line surface. The rest of this manual treats them interchangeably and refers to the binary as `clautolisp`.

3 Modes of Operation

`clautolisp` has three mutually exclusive modes; the mode is determined by the operands. In every mode the runtime installs the core builtin function bindings before evaluation begins.

3.1 File mode

A non-option argument is treated as a path to an AutoLISP source file. The reader loads the file under the active dialect, the evaluator processes every form sequentially, and the binary exits once the file has been fully consumed.

```
clautolisp drawing-tools.lsp
clautolisp --bricscad path/to/script.lsp
```

3.2 Expression mode

The `-x EXPRESSION` option evaluates the supplied string as if it were the entire source of a file, then exits.

```
clautolisp -x "(setq x 7) (+ x 3)"
clautolisp --bricscad -x "(princ (atof \"0x1p4\"))"
```

The expression is read with the source name `<-x>`, which appears in diagnostic messages.

3.3 Interactive REPL

With no `FILE` and no `-x`, `clautolisp` drops into an interactive read-eval-print loop on standard input.

- Primary prompt: ~_$ ~
- Continuation prompt (printed when an expression is incomplete): three spaces.
- Each balanced top-level form is evaluated and the result is printed using the AutoLISP value formatter (the same formatter used by `prin1`).

- A reader or runtime error is reported as a one-line diagnostic; the REPL then re-prompts.
- Closing standard input (Ctrl-D on a fresh line) ends the REPL.

The REPL accepts multi-line input: as long as the parser reports `:unexpected-eof`, the REPL keeps reading continuation lines and appends them to the in-progress form before retrying.

4 Options

All options begin with two dashes except for `-x`. They may appear in any order before, after, or interleaved with the `FILE` argument; the last occurrence of an option wins. `--` is **not** a recognised end-of-options marker; the runtime forwards every CLI argument to the parser.

4.1 Dialect selection

The dialect descriptor controls reader strictness, host-specific permissive parsing, and the runtime knobs documented in `autolisp-spec`. The default is the conservative `strict` profile.

`--dialect NAME` Select the named dialect. NAME is one of `strict`, `autocad-2026`, `bricscad-v26`.

`--strict` Equivalent to `--dialect strict`.

`--autocad` Equivalent to `--dialect autocad-2026`.

`--bricscad` Equivalent to `--dialect bricscad-v26`.

`--clautolisp` Equivalent to `--dialect clautolisp`.

The `clautolisp` dialect includes ‘clautolisp extensions‘ (eg. variadic functions).

Whatever the dialect selected, all the operators are available. Only those that are not specifically inside the set defined by the dialect produce warnings when compiled and when used.

Unknown dialect names produce a diagnostic on standard error and exit code 1.

4.2 Input selection

`-x EXPRESSION` Evaluate EXPRESSION instead of reading a file. Implies expression mode. Must be followed by exactly one argument.

4.3 REPL behaviour

`--quiet` Suppress the REPL banner; only the prompt is shown. No effect outside REPL mode.

4.4 Host backend

The Host Abstraction Layer (HAL, Phase 8 of the implementation roadmap) decides where CAD-host effects land. The dialect descriptor controls the **language**; the host backend controls the **target**. Both axes are independent.

`--host NAME` Select the backend. Currently accepted: `mock` (default since 0.0.14): an in-memory deterministic CAD-database substitute capable of entget / ssget / getvar / getstring etc. `null`: every CAD-host operation signals `:host-not-supported` — useful for programs that should not touch a host at all. `live` is reserved for Phase 16.

`--mock-input PATH` Attach the file at PATH as the MockHost's prompt-stream. Lines are consumed in order by `getstring`, `getreal`, `getint`, `getpoint`, etc. so headless test rigs can feed deterministic answers to interactive scripts.

4.5 Colour output

`clautolisp` accents AutoLISP-symbol names in error messages, backtraces, and trace output with an ANSI colour when standard output is a terminal. The default is conservative (yellow on assumed-dark) and never fires when the output is being captured (pipe, file redirect, test harness).

`--no-color` Disable colour for this invocation. Equivalent to setting `$NO_COLOR` to any non-empty value (see `https://no-color.org`).

`$NO_COLOR` Universal no-color convention; honoured equivalently to `--no-color`.

`$CLAUTOLISP_SYMBOL_FOREGROUND=NAME` Force the foreground colour for printed AutoLISP symbols. Accepted names (case and whitespace insensitive): `black`, `red`, `green`, `yellow`, `blue`, `magenta`, `cyan`, `white` and

their `bright-` prefixed variants (`bright-black` is also `grey` / `gray`). `default` / `none` / `off` leaves the foreground axis bare.

`$CLAUTOLISP_SYMBOL_BACKGROUND=NAME` Force the background colour for printed AutoLISP symbols. Same vocabulary as the foreground sibling; either or both env vars may be set. Both win over the automatic luminance probe so the user gets a deterministic look without depending on terminal detection.

`$CLAUTOLISP_BACKGROUND=dark|light` Override the terminal-background guess without specifying a colour. `dark` keeps the yellow accent; `light` flips it to blue.

`$CLAUTOLISP_PROBE_OSC11` Opt-in to the active OSC 11 terminal query (spawns `/bin/sh` + `stty` + `dd` against `/dev/tty`). Off by default because on some platforms — notably macOS arm64 — the helper subprocess receives SIGTTOU and hangs CLI startup. Safe to set on most Linux desktops and iTerm2 / Terminal.app launched from a normal shell.

Example: yellow symbols on a dim-blue background, deterministically, no probing:

```
export CLAUTOLISP_SYMBOL_FOREGROUND=bright-yellow
export CLAUTOLISP_SYMBOL_BACKGROUND=blue
clautolisp -x '(foreach x t (princ x))'
```

4.6 Informational

`--version` Print `clautolisp <version>` on standard output and exit 0. The version is the value of `clautolisp.tools.clautolisp:*version*` (format `MAJOR.MINOR.DEVELOP`).

`--help` Print a short usage summary and exit 0.

5 Banner

In REPL mode (and only in REPL mode) `clautolisp` prints a one-line banner before the first prompt:

```
clautolisp 0.0.9 - REPL (STRICT dialect, null-host host) - Ctrl-D to exit.
```

The version string, dialect name, and host backend name reflect the current build and the selected `--dialect` / `--host`. Pass `--quiet` to suppress the banner.

6 Exit Codes

`clautolisp` uses the following exit codes. They are stable and may be relied upon by shell scripts.

`0` Successful completion (file or `-x` ran without error; REPL exited cleanly via end-of-input; `--help` / `--version` printed and exited; an AutoLISP `exit` / `quit` termination was raised cleanly).

`1` A runtime error escaped the AutoLISP error-handling layer, or the option parser rejected an unknown flag.

`2` The requested file could not be opened (caught Common-Lisp `file-error`).

Within the REPL, individual evaluation errors are reported but do **not** terminate the loop; the binary will still exit 0 once stdin closes.

7 Errors and Diagnostics

- Reader errors are produced by the AutoLISP reader and surface as `simple-error` conditions whose printable form embeds a structured `diagnostic` (severity, code, message, source span). They are printed to standard error in batch modes and to standard error in REPL mode while the loop continues.
- Runtime errors are signalled as `autolisp-runtime-error` with a symbolic `code` (e.g. `UNDEFINED-FUNCTION`, `UNBOUND-VARIABLE`, `TYPE-ERROR`) and a human-readable message.
- Termination conditions raised by `exit` / `quit` (`autolisp-termination`) surface as a one-line notice on standard error and exit cleanly.

The exact set of error codes lives in the runtime; see `autolisp-runtime/source/api.lisp` for the canonical mapping.

8 Examples

```
# Print a literal symbol.
$ clautolisp -x "(print 'hi)"
HI

# Evaluate a small program under the BricsCAD V26 dialect.
$ clautolisp --bricscad -x "(setq x 1) (+ x 2)"

# Run a script file under the AutoCAD 2026 dialect.
$ clautolisp --autocad some-script.lsp

# Interactive REPL, default dialect.
$ clautolisp
clautolisp 0.0.1 - REPL (STRICT dialect) - Ctrl-D to exit.
_$ (setq xs (list 1 2 3))
(1 2 3)
_$ (length xs)
3
_$ ^D

# Interactive REPL with no banner.
$ clautolisp --quiet

# Show the version and exit.
$ clautolisp --version
clautolisp 0.0.1
```

9 Building the Executable

From the `clautolisp/` subproject root:

```
make build-clautolisp-sbcl   # produces tools/clautolisp/bin/clautolisp-sbcl
make build-clautolisp-ccl    # produces tools/clautolisp/bin/clautolisp-ccl
```

Both targets reuse the `read-autolisp` build harness. The SBCL build sets `:save-runtime-options t` so the saved core forwards every command-line argument to `clautolisp` rather than letting SBCL intercept `--help` or other runtime flags.

10 Versioning

The version string is stored in `clautolisp/tools/clautolisp/source/version.lisp` as `*version*`. The format is `MAJOR.MINOR.DEVELOP`. The DEVELOP counter is bumped on every change that touches clautolisp source code; the running binary's `--version` output is therefore a stable identifier of the build.

11 clautolisp Extensions

The core builtin registry installed by `clautolisp` is a **superset** of the operators listed in the local AutoLISP / Visual LISP specification draft. The extras documented in this section are convenience extensions and compatibility helpers that real-world AutoLISP code relies on without going through the spec process.

Code that targets the strict shared standard (the AutoCAD/BricsCAD intersection) MUST NOT depend on the operators below; tests written against `autolisp-test/harness/inventory.sexp` are intentionally restricted to the spec, and the inventory does not list these names.

11.1 Compositional list accessors (`Cxxxr` / `Cxxxxr` family)

The local specification documents `CAR`, `CDR` and `CADR`. Beyond those, `clautolisp` also installs:

two-level `CAAR`, `CDAR`, `CDDR`

three-level `CAAAR`, `CAADR`, `CADAR`, `CADDR`, `CDAAR`, `CDADR`, `CDDAR`, `CDDDR`

four-level `CAAAAR`, `CAAADR`, `CAADAR`, `CAADDR`, `CADAAR`, `CADADR`, `CADDAR`, `CADDDR`, `CDAAAR`, `CDAADR`, `CDADAR`, `CDADDR`, `CDDAAR`, `CDDADR`, `CDDDAR`, `CDDDDR`

Each composition delegates to `CAR` / `CDR` so a non-list argument anywhere along the chain still surfaces a CAR-shaped or CDR-shaped runtime error under the proper operator name.

```
$ clautolisp -x "(cadddr '(a b c d e))"
D
```

11.2 ERROR

`(error MESSAGE-OR-STRING ...)` raises an AutoLISP-visible runtime error with code `:user-error` and the supplied message. Modeled after the common Visual LISP idiom of calling `(error ...)` to abort with a user-facing message, and catchable with `vl-catch-all-apply`.

```
$ clautolisp -x "(vl-catch-all-error-message
                   (vl-catch-all-apply 'error '(\"oops\")))"
"oops"
```

11.3 LCM, LOGXOR

Convenience numerics that complement the in-spec `GCD`, `LOGAND` and `LOGIOR`. `LCM` takes two or more integers and returns their least common multiple; `LOGXOR` takes two or more integers and returns their bitwise exclusive-or.

```
$ clautolisp -x "(lcm 4 6)"
12
$ clautolisp -x "(logxor 12 10)"
6
```

11.4 VL-BT — full backtrace

The local specification reserves `VL-BT` as a Visual LISP backtrace operator. `clautolisp` implements it as a real backtrace printer rather than a stub: `(vl-bt)` prints the AutoLISP call stack at the current evaluation point, one line per frame, most recent first.

The runtime tracks the stack through `*autolisp-call-stack*`, a dynamically-bound list of evaluator frames pushed by `autolisp-eval` and by every SUBR / USUBR call. Frame kinds are `:eval`, `:special-op`, `:subr` and `:usubr`.

11.5 VL-CATCH-ALL-ERROR-STACK — captured backtrace

`(vl-catch-all-error-stack OBJECT)` is a clautolisp-only extension companion to the standard `vl-catch-all-error-message`. Given a catch-all error object, it returns the AutoLISP call-stack snapshot captured at the time the underlying `autolisp-runtime-error` was raised. Each frame is a cons `(KIND . PAYLOAD)`, where `PAYLOAD` is either the form being evaluated (for `:eval` / `:special-op`) or the cons `(NAME . ARGS)` of the function call (for `:subr` / `:usubr`).

This is the operator the `autolisp-test` harness calls to render an AutoLISP backtrace inside a FAIL detail when a tested function raises. AutoCAD and BricsCAD do not expose the captured stack, so the harness gracefully falls back to message-only reporting on those hosts.

```
$ clautolisp -x "(setq c (vl-catch-all-apply 'car '(7)))
                 (length (vl-catch-all-error-stack c))"
;; one or more frames captured at the time CAR was rejected
3
```

Code that needs to remain portable should test for the operator before calling it:

```
(if (boundp 'vl-catch-all-error-stack)
    (vl-catch-all-error-stack object)
    nil)
```

11.6 Source

The implementation of every operator listed above lives in:

- `clautolisp/autolisp-builtins-core/source/api.lisp` — registration and per-builtin code.
- `clautolisp/autolisp-builtins-core/README.org` — module-level rationale, kept in sync with this manual section.
- `clautolisp/autolisp-runtime/source/api.lisp` — backtrace tracking (`*autolisp-call-stack*`, `current-autolisp-call-stack`, the call-stack slot on `autolisp-runtime-error`, and the `autolisp-catch-all-error-call-stack` accessor).

12 See Also

- `../../autolisp-spec/` — AutoLISP / Visual LISP language reference, including dialect divergences and the normative source for builtin semantics.
- `PLAN.md` — Implementation roadmap and per-phase completion notes.
- `clautolisp/autolisp-reader/` — Reader subsystem (lexer, parser, diagnostics, dialect descriptors).

- `clautolisp/autolisp-runtime/` — Runtime object model and evaluator.
- `clautolisp/autolisp-builtins-core/` — Core builtin function registry (the layer that supplies `print`, `+`, `atof`, etc. consumed by the CLI).
- `../../autolisp-test/` — Conformance test suite. Uses `vl-catch-all-error-stack` and `vl-bt` to render AutoLISP backtraces inside FAIL details when a tested function raises on clautolisp.